B.Tech. BCSE497J - Project-I

AI CHESS BOT USING PYTORCH, FASTAPI, AND DEEP LEARNING

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

23BCE0312 MATTA SAI SANTHOSH

23BCE2140 Siddareddy Reddy Srinivas

23BCE2254 Meesaraganga Vishnu Vardhana Raju

Under the Supervision of

Dr. Sharief Basha S

Associate Professor Sr

School of Computer Science and Engineering (SCOPE)



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

DECLARATION

We hereby declare that the project entitled **"AI Chess Bot Using PyTorch, FastAPI, and Deep Learning"**, submitted for the award of the degree of Bachelor of Technology in Computer Science and Engineering at Vellore Institute of Technology, is a record of project work carried out by us under the supervision of Dr. Sharief Basha S.

We further declare that the work reported in this project has not been submitted, either in part or in full, for the award of any other degree or diploma at this institute or any other institute or university.

Place: Vellore

Date:

Signatures of the Candidates:

1. Matta Sai Santhosh – 23BCE0312

2. Siddareddy Reddy Srinivas – 23BCE2140

3. Meesaraganga Vishnu Vardhana Raju – 23BCE2254

CERTIFICATE

This is to certify that the project entitled **"AI Chess Bot Using PyTorch, FastAPI, and Deep Learning"**, submitted by Matta Sai Santhosh (23BCE0312), Siddareddy Reddy Srinivas (23BCE2140), and Meesaraganga Vishnu Vardhana Raju (23BCE2254), of the School of Computer Science and Engineering, Vellore Institute of Technology, for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of project work carried out under the supervision of Dr. Sharief Basha S during the Fall Semester 2026–2027, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted, either in part or in full, for the award of any other degree or diploma at this institute or any other institute or university. The project fulfils the applicable academic requirements and, in the opinion of the supervisor, meets the necessary standards for submission.

Place: Vellore

Date:

Signature of the Guide: _____

Examiner(s): _____

ACKNOWLEDGEMENTS

We express our sincere gratitude to the management of Vellore Institute of Technology (VIT) for providing us with the opportunity and resources to undertake this project. The academic environment and technical infrastructure provided by the institution have supported our learning and practical development.

We extend our thanks to the School of Computer Science and Engineering (SCOPE) for providing an environment that encourages technical exploration, innovation, and project-based learning.

We express our profound appreciation to our project guide, **Dr. Sharief Basha S**, for his guidance, valuable feedback, and support throughout the development of this project. His guidance helped us understand the design and implementation challenges involved in integrating deep learning with a web-based application.

We also acknowledge the developers and maintainers of the open-source technologies used in this project, including PyTorch, FastAPI, python-chess, NumPy, and related tools. Their documentation and software resources supported the implementation of the system.

Finally, we thank our teammates for their collaboration and contributions toward the development of the AI Chess Bot.

Matta Sai Santhosh

Siddareddy Reddy Srinivas

Meesaraganga Vishnu Vardhana Raju

ABSTRACT

1. INTRODUCTION

1.1 Background

1.2 Motivations

1.3 Scope of the Project

2. PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

2.2 Research Gap

2.3 Objectives

2.4 Problem Statement

2.5 Project Plan

3. TECHNICAL SPECIFICATION

3.1 Requirements

3.1.1 Functional Requirements

3.1.2 Non-Functional Requirements

3.2 Feasibility Study

3.2.1 Technical Feasibility

3.2.2 Economic Feasibility

3.2.3 Social Feasibility

3.3 System Specification

3.3.1 Hardware Specification

3.3.2 Software Specification

4. DESIGN APPROACH AND DETAILS

4.1 System Architecture

4.2 Design

4.2.1 Data Flow Diagram

4.2.2 Use Case Diagram

4.2.3 Class Diagram

4.2.4 Sequence Diagram

5. METHODOLOGY AND TESTING

5.1 Theoretical Foundations

5.1.1 Artificial Intelligence in Chess

5.1.2 Convolutional Neural Networks

5.1.3 Residual Neural Networks

5.1.4 Policy Networks and Move Prediction

5.2 Methodological Procedure

5.2.1 Chessboard Representation and Encoding

5.2.2 Neural Network Inference

5.2.3 Legal-Move Filtering

5.2.4 AI Move Selection

5.3 Implementation Workflow

5.4 Module Description

5.4.1 Frontend Module

5.4.2 FastAPI Backend Module

5.4.3 Chess Game Management Module

5.4.4 Board Encoding Module

5.4.5 Neural Network Module

5.4.6 Move Selection Module

5.4.7 Model Loading and Inference Module

5.5 Testing

5.5.1 Functional Testing

5.5.2 API Testing

5.5.3 Move Validation Testing

5.5.4 Integration Testing

6. PROJECT DEMONSTRATION

6.1 Interactive Application

6.2 Chessboard Initialization and User Interaction

6.3 AI Move Generation

6.4 Game-State Handling

6.5 Experiment Screenshots

7. RESULT AND DISCUSSION

7.1 Functional Testing Results

7.2 Inference Performance

7.3 Legal-Move Validation Results

7.4 Game Scenario Evaluation

7.5 Combined System Evaluation

7.6 Quantitative Results

7.7 Qualitative Results

7.8 Cost Analysis

8. CONCLUSION

8.1 Conclusion

8.2 Future Work

9. REFERENCES

[9] Python Software Foundation, “Threading — Thread-based parallelism,” Python 3 Library Reference. [Online]. Available: https://docs.python.org/3/library/threading.html. [Accessed: Oct. 27, 2026].

[10] Hugging Face, “Downloading files,” Hugging Face Hub documentation. [Online]. Available: https://huggingface.co/docs/huggingface_hub/guides/download. [Accessed: Oct. 27, 2026].

[11] V. Raju, “Chess-Bot,” GitHub repository. [Online]. Available: https://github.com/VishnuRaju06/Chess-Bot. [Accessed: Oct. 27, 2026].

[12] V. Raju, “Chess-Bot,” Hugging Face Space. [Online]. Available: https://huggingface.co/spaces/vishnuraju06/Chess-Bot. [Accessed: Oct. 27, 2026].

APPENDIX A – SAMPLE CODE

List of Figures

Figure No.	Title	Page No.
4.1	System Architecture of the AI Chess Bot	25
4.2	Data Flow Diagram of the AI Chess Bot	26
4.3	Use Case Diagram of the AI Chess Bot	27
4.4	Class Diagram of the AI Chess Bot	28
4.5	Sequence Diagram of the AI Chess Bot	29
5.1	Workflow of the AI Chess Bot	35
5.2	Chessboard Encoding Representation	32
5.3	Architecture of the Residual Policy Network	33
5.4	Legal-Move Filtering and AI Move Selection	34
6.1	Initial Chessboard Interface	42
6.3	Chessboard After an AI Move	43

List of Tables

Table No.	Title	Page No.
2.1	Summary of Relevant Technologies and Approaches	16
2.2	Project Development Plan	18
3.1	Functional Requirements	20
3.2	Non-Functional Requirements	21
3.3	Hardware Specification	23
3.4	Software Specification	23
5.1	Module Description of the AI Chess Bot	36
5.2	Functional Testing Test Cases	37
5.3	API Testing Test Cases	37
5.4	Move Validation Testing Results	38
5.5	Integration Testing Results	39
7.1	Functional Testing Results	44
7.2	AI Move Response-Time Measurements	45
7.3	Legal-Move Validation Results	46
7.4	Game Scenario Evaluation	46
7.5	Cost Analysis	47

List of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSS	Cascading Style Sheets
FEN	Forsyth–Edwards Notation
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
ML	Machine Learning
NN	Neural Network
ReLU	Rectified Linear Unit
REST	Representational State Transfer
RAM	Random Access Memory
TCP	Transmission Control Protocol
UCI	Universal Chess Interface
UI	User Interface
URL	Uniform Resource Locator
XML	Extensible Markup Language

Symbols and Notations

Symbol / Notation	Description
(B)	Chessboard state
(s)	Current state of the chessboard
(a)	Candidate chess move
P(a | s)	Probability of selecting move (a) in state (s)
(X)	Input tensor representing the chessboard
18 × 8 × 8	Dimensions of the encoded chessboard tensor
(N)	Number of test cases
T i	Response time for test case (i)
T avg	Average response time
S valid	Valid-move success rate
R illegal	Illegal-move rejection rate
(tau)[τ]	Temperature parameter for AI move selection
(k)	Number of candidates retained by top-k selection
(sum)[Σ]	Summation operator
(100%)	Conversion of a ratio into a percentage
FEN	Forsyth–Edwards Notation for representing a chess position
UCI	Universal Chess Interface notation used to represent chess moves

ABSTRACT

Chess is a strategic board game that requires logical reasoning, planning, and decision-making. Developing an artificial intelligence system capable of selecting appropriate chess moves is an important application of machine learning and deep learning. This project, titled "AI Chess Bot Using PyTorch, FastAPI, and Deep Learning," focuses on developing an interactive chess application in which a human player can play against an AI-based opponent.

The proposed system integrates a web-based user interface with a Python backend and a deep learning model. The frontend is developed using HTML, CSS, and JavaScript to provide an interactive chessboard and enable users to make moves. FastAPI is used to handle communication between the frontend and backend, while the Python Chess library manages the chessboard state, validates moves, and maintains the rules of the game. PyTorch is used to implement and execute the neural network responsible for evaluating possible moves.

The chessboard position is converted into a numerical tensor of dimensions $18 \times 8 \times 8$, which serves as the input to the neural network. The model uses a convolutional layer followed by residual blocks and a policy head to generate scores for possible chess moves. These scores are processed to identify promising candidate moves. The system then filters the predicted moves against the legal moves available in the current position to ensure that the AI selects a valid move. Temperature-based sampling and top-k selection can be used during move selection to control the variety of AI responses.

The application follows a modular architecture consisting of the frontend interface, API layer, chess game manager, board encoding module, neural network, move selection component, and model inference mechanism. This separation of responsibilities improves maintainability and supports testing of individual components as well as the complete application.

This project implements a browser-based chess application that connects a PyTorch policy network to a FastAPI service and the python-chess rules engine. A player can choose White or Black, play through an interactive board, and receive AI moves selected from the position's legal moves. The application maintains a separate game state for each game identifier. This report documents the implemented architecture, verified gameplay observations, and limitations; no formal response-time benchmark or controlled load test is claimed.

Keywords: Artificial Intelligence, Deep Learning, Chess Bot, PyTorch, FastAPI, Neural Network, Move Prediction.

CHAPTER 1

1. INTRODUCTION

1.1 Background

Artificial Intelligence (AI) is an important field of computer science that enables machines to perform tasks that normally require human intelligence, such as problem-solving, decision-making, pattern recognition, and learning. One of the well-known applications of AI is chess, a strategic board game that requires players to analyze different positions, predict possible moves, and develop effective strategies. Due to its large number of possible board positions and move combinations, chess provides a challenging environment for developing and evaluating intelligent systems.

Traditional chess programs generally rely on search algorithms, predefined rules, and evaluation functions to determine suitable moves. With advances in machine learning and deep learning, neural networks have become another approach for learning patterns from chess positions and predicting promising moves. Convolutional Neural Networks (CNNs) can process structured board representations, while residual connections help deeper neural networks learn complex patterns more effectively.

This project, titled “AI Chess Bot Using PyTorch, FastAPI, and Deep Learning,” focuses on developing an interactive application that allows a human player to play chess against an AI-based opponent. The system uses PyTorch to execute a neural network that predicts promising moves, FastAPI to provide communication between the frontend and backend, and the Python Chess library to manage the board state and enforce chess rules. The frontend, developed using HTML, CSS, and JavaScript, provides an interactive chessboard through which users can make moves and observe the AI's responses.

The chessboard is converted into a numerical representation before being processed by the neural network. The model produces scores for possible moves, and the move-selection component uses these predictions to identify a suitable candidate. Legal-move filtering ensures that the selected move follows the rules of chess. Through the integration of deep learning, backend services, and web technologies, the project demonstrates the practical application of AI in an interactive game environment.

1.2 Motivations

The main motivation behind this project is to explore how deep learning techniques can be applied to strategic decision-making problems. Chess requires evaluating a current position, considering alternative moves, and selecting an action from several possibilities. These characteristics make it a useful application for studying neural networks and intelligent systems.

Another motivation is to integrate machine learning with software development technologies. Building a chess bot involves more than creating a neural network; the model must also interact with a game engine, receive board information, generate predictions, and return valid moves to the user interface. Combining PyTorch, FastAPI, Python Chess, and web technologies provides practical experience in designing a complete AI-powered application.

The project also aims to make interaction with an AI model simple and accessible through a graphical interface. Instead of requiring users to interact directly with Python code or

command-line tools, the application provides a chessboard through which users can play moves and observe the system's decisions.

The major motivations are:

- To understand the application of deep learning to chess move prediction.
- To explore convolutional and residual neural network architectures.
- To integrate a trained neural network with a chess game management system.
- To develop communication between a web frontend and a Python backend using FastAPI.
- To ensure that AI-generated moves comply with the legal rules of chess.
- To evaluate the application's functionality, reliability, and response time through testing.

Overall, the project provides an opportunity to apply concepts from artificial intelligence, deep learning, and web application development in a single practical implementation.

1.3 Scope of the Project

The scope of this project is to design and implement a web-based chess application that enables a human player to play against a deep learning-based AI opponent. The application combines a graphical interface, backend API services, chess rule management, and neural network inference.

The frontend provides the chessboard and supports user interaction. The backend receives requests from the interface and manages the current game state. The Python Chess library checks the legality of moves and maintains the rules of the game. The board encoding module converts the current position into a numerical tensor suitable for the neural network, and the PyTorch model generates predictions for possible moves. The move-selection module processes these predictions, filters illegal moves, and selects a valid AI response.

The project covers the following areas:

- **User interface:** Displaying the chessboard and allowing the user to interact with the game.
- **Backend communication:** Processing game requests through FastAPI.
- **Game management:** Maintaining board positions, validating moves, and tracking the game state.
- **Board encoding:** Converting the chess position into an $18 \times 8 \times 8$ input tensor.
- **Neural network inference:** Generating move predictions using the PyTorch model.
- **Move selection:** Applying candidate selection and legal-move filtering to produce a valid move.

- **Testing and evaluation:** Checking functionality, API responses, move validity, integration, and response time.

The project primarily focuses on integrating and executing the AI model within a functional chess application rather than developing a complete commercial chess engine. Its performance depends on the trained model, the quality of its learned patterns, and the available computing resources.

Future enhancements may include additional model training, stronger position evaluation, improved gameplay features, move-history visualization, difficulty settings, and more extensive performance evaluation. These enhancements could improve the application's usability and the quality of its chess-playing decisions.

CHAPTER 2

2. PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

Artificial Intelligence has been widely applied to chess to develop systems that can analyse board positions and select suitable moves. Traditional chess programs mainly use search algorithms and evaluation functions to explore possible moves and estimate the quality of a position. With the development of deep learning, neural networks have become an important approach for learning patterns from board positions and predicting promising moves.

Convolutional Neural Networks (CNNs) are useful for processing structured representations of chessboards because they can learn spatial patterns from input data. Residual neural networks extend this approach by using skip connections, which help information flow through deeper network architectures. Policy networks use the learned representation of a position to assign scores or probabilities to candidate moves.

Frameworks such as PyTorch provide the tools required to build and execute neural networks. FastAPI supports the development of backend APIs that connect machine learning models with applications. The Python Chess library provides functionality for representing board positions, validating moves, and managing game rules. Combining these technologies makes it possible to develop a chess application that integrates neural network inference with legal move validation and an interactive interface.

The proposed project brings together these concepts in a web-based chess application. The neural network predicts promising moves, while the game-management component ensures that the selected moves follow the rules of chess. The frontend provides an accessible way for users to interact with the system.

Table 2.1: Summary of Relevant Technologies and Approaches

Technology / Approach	Purpose	Application in the Project
Artificial Intelligence	Enables machine-based decision-making	Provides the foundation for the chess-playing system
Deep Learning	Learns patterns from data	Supports chess move prediction
Convolutional Neural Network	Processes structured input representations	Extracts features from the encoded chessboard
Residual Neural Network	Uses skip connections in deeper networks	Forms part of the policy network architecture
PyTorch	Deep learning framework	Loads and executes the neural network
FastAPI	Backend API framework	Handles communication between the frontend and backend

Python Chess	Chess rules and board management library	Maintains the game state and validates moves
HTML, CSS, JavaScript	Web interface technologies	Displays the chessboard and supports user interaction

2.2 Research Gap

Traditional chess engines can provide strong gameplay by combining search techniques with carefully designed evaluation functions. However, implementing a complete chess engine can require substantial algorithmic complexity and computational resources. A learning-based approach offers an alternative for predicting promising moves from board representations.

A neural network alone does not guarantee that its predicted move is legal. Its output must be integrated with a chess rules system to ensure that the final move is valid for the current board position. Similarly, a trained model requires an application layer that manages game state, accepts user actions, performs inference, and returns results to the interface.

The project addresses these integration requirements by combining a deep learning policy network with a chess game manager and a web API. The system uses the model to generate move predictions and applies legal-move filtering before returning an AI move. This design demonstrates how machine learning inference and rule-based validation can operate together in a practical application.

The project is an implementation and integration effort rather than a claim that the proposed approach outperforms established chess engines. Its effectiveness must be assessed through functional tests, move-validity checks, response-time measurements, and gameplay evaluation.

2.3 Objectives

The primary objective of this project is to develop an interactive chess application that allows a human player to play against an AI opponent powered by a deep learning model.

The specific objectives are:

1. To develop a web-based chess interface using HTML, CSS, and JavaScript.
2. To implement backend communication using FastAPI.
3. To manage chessboard positions and enforce chess rules using the Python Chess library.
4. To encode the current board position into an $18 \times 8 \times 8$ numerical tensor.
5. To integrate a PyTorch neural network that predicts promising chess moves.
6. To process the model's predictions and select a suitable candidate move.
7. To filter illegal moves and ensure that the selected AI move is valid.
8. To maintain the game state throughout human and AI interactions.
9. To test the functionality, API communication, move validation, and integration of the application.

10. To evaluate the system using response-time measurements and representative game scenarios.

2.4 Problem Statement

Developing an interactive AI chess application requires the integration of multiple components, including a user interface, a chess rules engine, a deep learning model, and a communication mechanism between the frontend and backend. These components must work together to process human moves, represent the current chessboard, generate AI predictions, validate the selected move, and update the game state.

The main problem addressed by this project is how to integrate a trained deep learning policy network into a functional chess application while maintaining correct chess rules and reliable communication between system components.

The input to the system is the current chessboard position and the user's move. The application updates the board state, converts the position into a numerical representation, and passes it to the neural network. The model generates scores for possible moves, and the move-selection component processes the predictions. The chess rules library then ensures that the chosen move is legal before the updated position is returned to the frontend.

The expected outcome is a functional application in which users can play chess against an AI-based opponent through a web interface. The system should correctly process valid user moves, generate valid AI responses, maintain the game state, and provide reliable communication between the frontend and backend.

2.5 Project Plan

The project is divided into a sequence of development activities to support systematic implementation, integration, and evaluation. The activities begin with understanding the requirements and reviewing the technologies, followed by interface development, backend implementation, model integration, testing, and report preparation.

Table 2.2: Project Development Plan

Phase	Activity	Expected Outcome
Phase 1	Requirement analysis and project planning	Defined project scope and functional requirements
Phase 2	Literature review and technology selection	Understanding of relevant AI and software technologies
Phase 3	Frontend development	Interactive chessboard interface
Phase 4	Backend and game-management implementation	API communication and chess rule handling
Phase 5	Board encoding and neural network integration	Model input preparation and move prediction

Phase 6	AI move selection and legal-move validation	Valid AI responses to user moves
Phase 7	System integration and testing	Integrated application with documented test results
Phase 8	Performance evaluation and report preparation	Evaluation results and final project documentation

The development process follows a modular approach so that individual components can be implemented and tested before complete system integration. The frontend, backend, game-management module, and neural network are connected progressively to identify integration issues early. Final testing focuses on the correctness of chess moves, API behaviour, game-state updates, and application responsiveness.

CHAPTER 3

3. TECHNICAL SPECIFICATION

3.1 Requirements

The technical requirements define the functions that the AI Chess Bot must perform and the quality attributes expected from the application. The requirements are divided into functional and non-functional requirements. Functional requirements describe the operations supported by the system, whereas non-functional requirements describe aspects such as reliability, usability, maintainability, and responsiveness.

3.1.1 Functional Requirements

Functional requirements specify the essential operations of the AI Chess Bot. The system must allow users to interact with a chessboard, submit moves, and receive AI-generated responses. It must maintain the current game state and ensure that moves follow the rules of chess.

The major functional requirements are listed below.

Table 3.1: Functional Requirements

Requirement ID	Functional Requirement	Description
FR1	Display Chessboard	The application shall display an interactive chessboard to the user.
FR2	Accept Human Moves	The system shall accept user moves through the interface.
FR3	Validate Moves	The game-management component shall check whether a move is legal.
FR4	Maintain Game State	The system shall maintain the current board position throughout the game.
FR5	Encode Board Position	The system shall convert the board position into the required numerical tensor representation.
FR6	Generate AI Predictions	The neural network shall produce scores for possible chess moves.
FR7	Select AI Move	The move-selection component shall process the model predictions and select a candidate move.
FR8	Filter Illegal Moves	The system shall ensure that the selected AI move is legal before applying it.
FR9	Update Interface	The application shall display the updated board after a valid move.
FR10	Handle Game State	The system shall identify and handle game-ending conditions supported by the game-management component.

3.1.2 Non-Functional Requirements

Non-functional requirements describe the expected quality and operational characteristics of the application. These requirements help ensure that the system remains usable, reliable, and maintainable during execution.

The major non-functional requirements are:

- **Usability:** The interface should be simple and understandable so that users can play chess without interacting directly with backend code.
- **Reliability:** The application should maintain a consistent game state and handle valid requests correctly.
- **Performance:** The system should process user requests and generate AI responses within a reasonable time under the available computing conditions.
- **Correctness:** The final AI move must be checked against the legal moves available in the current board position.
- **Maintainability:** The modular design should allow the frontend, backend, game-management component, and model-inference component to be maintained separately.
- **Compatibility:** The web interface should work in a compatible modern browser, while the backend should run in an environment that supports its Python dependencies.
- **Robustness:** The application should handle invalid moves or unsupported requests appropriately instead of corrupting the game state.

Table 3.2: Non-Functional Requirements

Requirement ID	Attribute	Description
NFR1	Usability	The interface should support straightforward interaction with the chessboard.
NFR2	Reliability	Game-state updates should remain consistent during normal operation.
NFR3	Performance	AI inference and API processing should be measured to assess responsiveness.
NFR4	Correctness	AI moves must be validated against the legal moves for the current position.
NFR5	Maintainability	Individual modules should be understandable and maintainable.
NFR6	Compatibility	The application should run in a supported browser and Python environment.
NFR7	Robustness	Invalid requests should be handled without causing unintended game-state changes.

3.2 Feasibility Study

A feasibility study evaluates whether the proposed project can be developed and operated using the available technologies and resources. The AI Chess Bot is examined from technical, economic, and social perspectives.

3.2.1 Technical Feasibility

The project is technically feasible because it uses established software technologies for frontend development, backend communication, chess rule management, and neural network inference. HTML, CSS, and JavaScript provide the user interface, while FastAPI handles communication with the Python backend. The Python Chess library manages the chessboard and validates moves, and PyTorch supports execution of the neural network.

The system is divided into independent modules, which simplifies implementation and debugging. The frontend communicates with the backend through API requests, and the backend coordinates game-state management with the model-inference process. A compatible Python environment and the required dependencies are necessary to execute the application.

The project's practical performance depends on the model, available hardware, dependency compatibility, and inference requirements. Therefore, technical feasibility should be confirmed by running the application and completing integration and performance tests in the target environment.

3.2.2 Economic Feasibility

The project can be developed primarily with open-source software, including Python, PyTorch, FastAPI, and the Python Chess library. The frontend uses standard web technologies, and development can be carried out using an existing computer and internet connection.

Consequently, the project does not necessarily require paid software licences. However, costs may arise from computing resources, internet usage, hosting, or optional cloud services if they are used. The actual cost depends on the deployment method and the resources required to run the model.

For an academic prototype developed and tested locally, the direct software cost can be low. Any cost estimates included in the final report should reflect the resources actually used.

3.2.3 Social Feasibility

The project provides an accessible way for users to interact with an AI-based chess application. It can support learning, experimentation, and exploration of the relationship between chess strategies and artificial intelligence.

The graphical interface reduces the need for users to understand the internal implementation. The project can also serve as an educational example of how deep learning models can be integrated into a complete software application.

The application should be presented as an educational and experimental tool. Its move predictions should not automatically be treated as equivalent to the decisions of a strong, established chess engine.

3.3 System Specification

System specifications describe the hardware and software resources required to develop, run, and test the AI Chess Bot. The actual requirements depend on the model size, inference configuration, and whether the application is executed locally or on a remote service.

3.3.1 Hardware Specification

The hardware environment must be capable of running the frontend, backend services, chess game-management code, and neural network inference. A computer with sufficient memory and processing capability can be used for local execution. A compatible GPU may help accelerate neural network operations, but its necessity depends on the model and runtime configuration.

Table 3.3: Hardware Specification

Component	Specification / Requirement
Processor	A modern multi-core CPU suitable for Python execution
RAM	Sufficient memory for the operating system, Python dependencies, and model
Storage	Sufficient free space for source code, dependencies, model weights, and logs
GPU	Optional; useful if supported and required for accelerated inference
Display	Monitor capable of displaying a modern web browser
Network	Internet connection if downloading model weights or installing dependencies is required

3.3.2 Software Specification

The software environment includes the operating system, programming language, frontend technologies, backend framework, chess library, and deep learning framework. The correct versions of the dependencies must be installed for successful execution.

Table 3.4: Software Specification

Software / Technology	Purpose
Python	Backend logic, game management, and model inference
PyTorch	Loading and executing the neural network
FastAPI	Providing backend API endpoints
Python Chess	Chessboard representation, legal-move validation, and game-state management

HTML	Structure of the web interface
CSS	Styling the chessboard and interface
JavaScript	User interaction and communication with the backend
Web Browser	Displaying and interacting with the application
Python Dependencies	Supporting the modules specified by the project configuration
Model Weights	Providing the trained parameters required for inference

The software should be configured according to the project's dependency file and execution instructions. If the model weights are downloaded during startup, the environment must also have access to the configured model source. Once the dependencies and model are available, the application can be launched and tested through the web interface.

CHAPTER 4

4. DESIGN APPROACH AND DETAILS

This chapter describes the design of the proposed AI Chess Bot, including its system architecture, data flow, user interactions, software components, and sequence of operations. The system combines a web-based frontend, a FastAPI backend, a chess game-management component, and a deep learning model implemented using PyTorch.

The design follows a modular approach in which each component performs a specific responsibility. The frontend manages user interaction, the backend processes requests, the chess-management component maintains the board state and validates moves, and the neural network predicts promising moves for the AI opponent. These components work together to provide an interactive chess-playing experience.

4.1 System Architecture

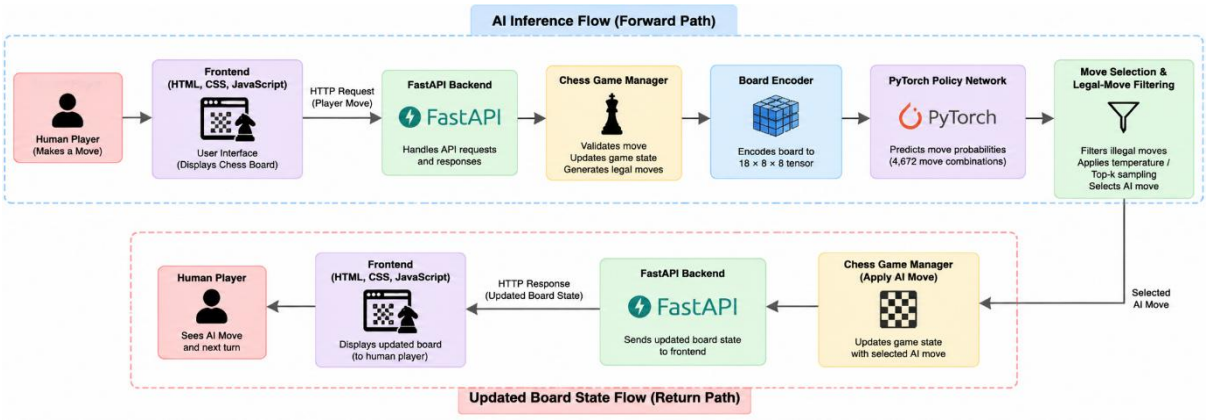
The system architecture illustrates the major components of the AI Chess Bot and the communication between them. The application is organized into a frontend layer, an API layer, a game-management layer, and a deep learning inference layer.

The frontend is developed using HTML, CSS, and JavaScript. It displays the chessboard, receives user actions, and presents the updated game state. The FastAPI backend receives requests from the frontend and coordinates the corresponding game operations.

The game-management component uses the Python Chess library to maintain the chessboard position and enforce chess rules. When an AI response is required, the current board position is converted into a numerical representation by the board-encoding component. The encoded representation is supplied to the PyTorch policy network, which produces scores for possible moves.

The move-selection component processes the model's predictions and selects a candidate move. Legal-move filtering ensures that the selected move is permitted in the current position. The game manager then updates the board, and the resulting game state is returned through the API to the frontend.

Figure 4.1: System Architecture of the AI Chess Bot



4.2 Design

The system design is represented using a Data Flow Diagram, Use Case Diagram, Class Diagram, and Sequence Diagram. Each diagram describes a different aspect of the application, from the movement of data to the relationships between components and the order of execution.

4.2.1 Data Flow Diagram (DFD)

A Data Flow Diagram represents how information moves through the AI Chess Bot. It identifies the external user, the major processing activities, and the data exchanged between the components.

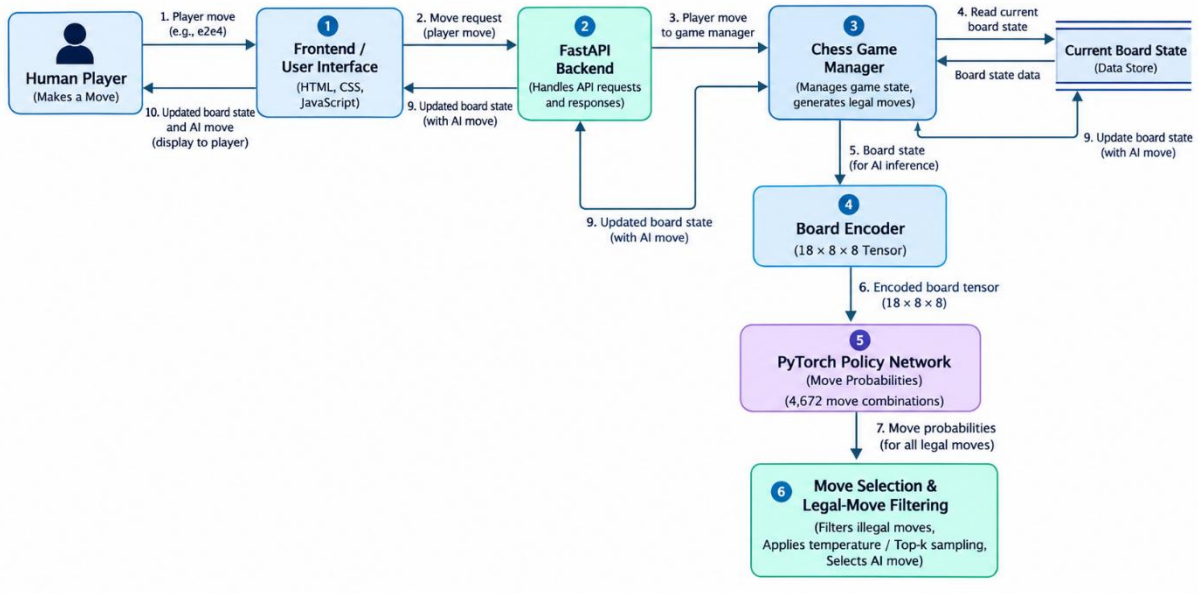
The human player interacts with the frontend by making a chess move. The application sends the move to the backend, where the game-management component checks its validity and updates the current board position. When an AI move is required, the current position is encoded and passed to the neural network for prediction. The move-selection component evaluates the predictions and selects a legal move. The updated board state is then returned to the frontend for display.

The principal processes represented in the DFD are:

1. Receive a human move from the frontend.
2. Validate the move and update the game state.
3. Encode the current board position.
4. Generate move predictions using the neural network.
5. Select and validate the AI move.
6. Return the updated game state to the user interface.

Figure 4.2: Data Flow Diagram of the AI Chess Bot

Figure 4.2: Data Flow Diagram (DFD) of the AI Chess Bot



4.2.2 Use Case Diagram

The Use Case Diagram describes the functionality available to the user and the interactions between the human player and the AI Chess Bot. The primary actor is the Human Player, who uses the application to play chess against the AI opponent.

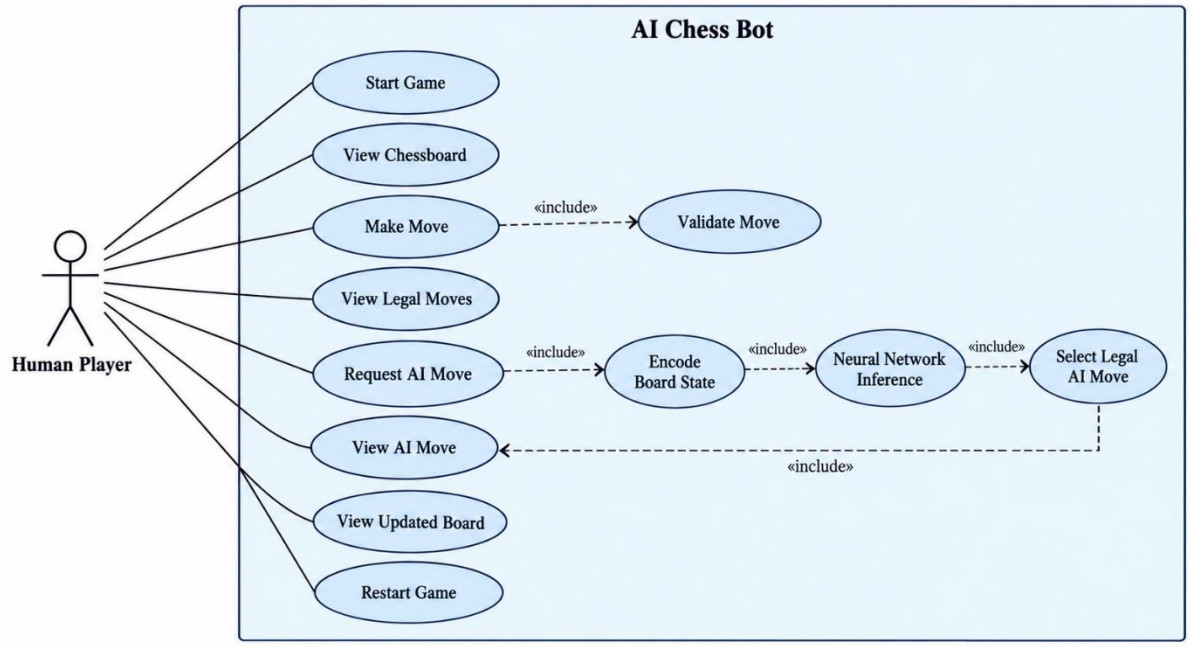
The user can view the chessboard, make a move, observe the AI's response, and follow the progress of the game. The application processes these actions, validates moves, updates the board, and handles game-ending conditions supported by the chess-management component.

The main use cases are:

- View the chessboard.
- Make a chess move.
- Validate the submitted move.
- Request an AI response through the normal gameplay flow.
- Display the AI move.
- View the updated board position.
- Handle game-ending conditions.

Figure 4.3: Use Case Diagram of the AI Chess Bot

Figure 4.3: Use Case Diagram of the AI Chess Bot



4.2.3 Class Diagram

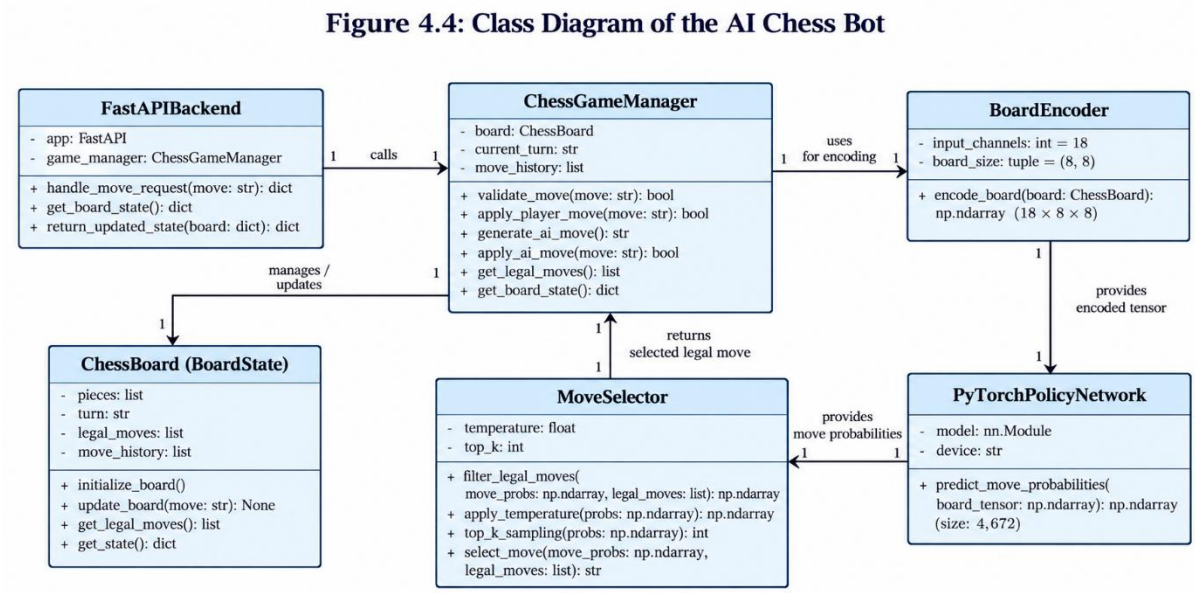
The Class Diagram represents the logical organization of the software components and their relationships. It helps explain how the frontend, API, game manager, board encoder, neural network, and move-selection logic contribute to the overall system.

The frontend communicates with the backend API to submit moves and receive updated game information. The API coordinates operations through the game manager. The game manager maintains the current board state and uses the chess rules library to validate moves. For AI move generation, the board encoder prepares the input representation, the neural network generates predictions, and the move-selection component processes the predictions before returning a valid move.

The principal logical components are:

- **Frontend:** Presents the chessboard and handles user interaction.
- **FastAPI Backend:** Receives and processes application requests.
- **Game Manager:** Maintains the board state and coordinates game operations.
- **Board Encoder:** Converts the board position into a numerical tensor.
- **Policy Network:** Generates scores for possible moves using PyTorch.
- **Move Selector:** Processes predictions and selects a candidate move.
- **Model Loader and Inference Support:** Makes the trained model available for prediction.

Figure 4.4: Class Diagram of the AI Chess Bot



4.2.4 Sequence Diagram

The Sequence Diagram illustrates the chronological order of interactions when a human player makes a move and the AI generates its response. It shows how requests and responses pass between the frontend, backend, game manager, board encoder, neural network, and move-selection component.

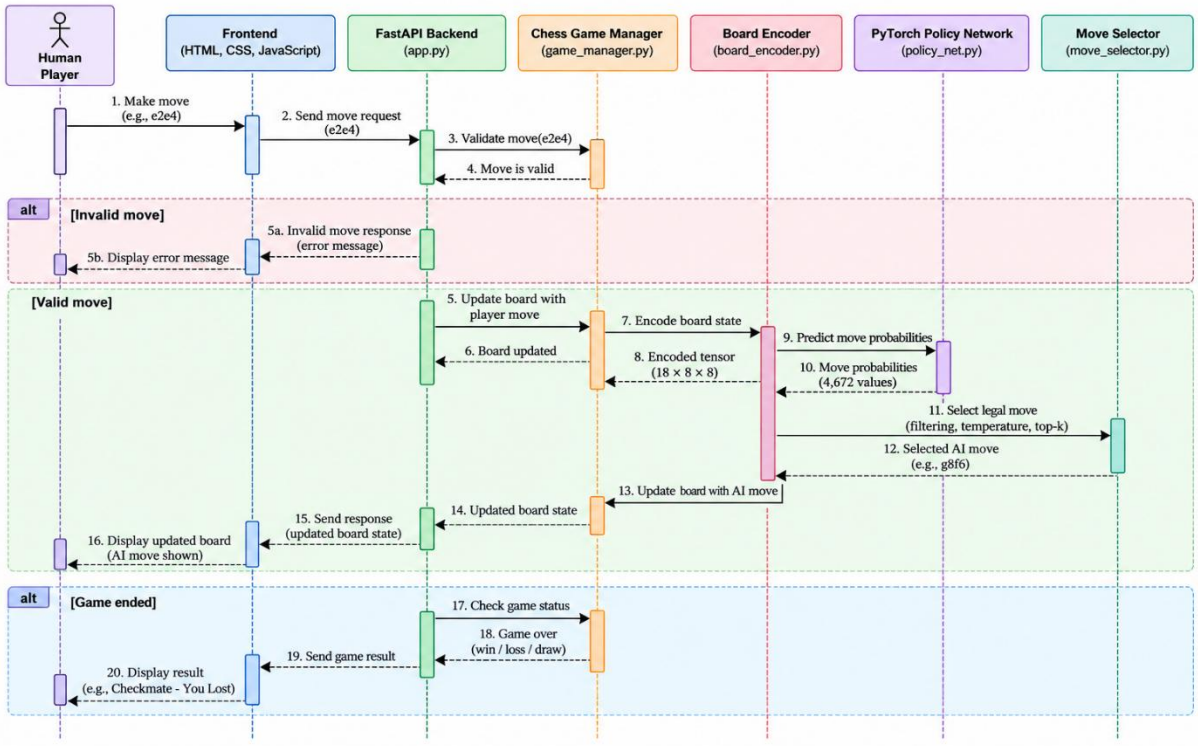
Initially, the human player makes a move through the frontend. The frontend submits the move to the FastAPI backend. The game manager validates the move and updates the board state. If the game continues and an AI response is required, the current position is encoded and passed to the neural network. The model generates predictions, and the move selector processes the output to choose a legal move. The game manager applies the AI move and prepares the updated game state. Finally, the backend returns the result to the frontend, which displays the updated chessboard.

The sequence consists of the following stages:

1. The Human Player submits a move through the frontend.
2. The frontend sends a request to the FastAPI backend.
3. The game manager validates the move and updates the board.
4. The board encoder generates the model input.
5. The policy network generates move predictions.
6. The move selector filters candidate moves and chooses a legal move.
7. The game manager applies the AI move and updates the game state.
8. The backend returns the updated state to the frontend.
9. The frontend displays the new chessboard position.

Figure 4.5: Sequence Diagram of the AI Chess Bot

Figure 4.5: Sequence Diagram of the AI Chess Bot



CHAPTER 5

5. METHODOLOGY AND TESTING

This chapter explains the methodology followed to develop the AI Chess Bot and describes the major modules involved in its implementation. The system integrates a web-based interface, a FastAPI backend, chess game management, board encoding, a deep learning policy network, and AI move selection. Each module performs a specific task, and the modules work together to provide an interactive chess-playing application.

The chapter also describes the testing procedures used to evaluate the correctness of user interactions, backend communication, chess move validation, model inference, and complete system integration. Testing is important to ensure that the application processes moves correctly and maintains a consistent game state throughout a chess game.

5.1 Theoretical Foundations

The AI Chess Bot combines concepts from artificial intelligence, deep learning, neural networks, and rule-based game management. Understanding these concepts helps explain how the system receives a chess position, generates move predictions, and selects an appropriate AI response.

5.1.1 Artificial Intelligence in Chess

Artificial Intelligence enables computer systems to perform tasks that involve decision-making and problem-solving. In chess, an intelligent system must process the current board position, identify possible moves, and select an action.

Traditional chess engines often use search algorithms and evaluation functions to analyze possible continuations. A learning-based approach uses a trained neural network to identify patterns in board positions and predict promising moves. The effectiveness of a learning-based system depends on its training data, model architecture, and move-selection strategy.

In this project, the neural network produces predictions for possible moves. The chess rules component validates the selected move before it is applied to the board. This combination allows the system to use learned predictions while maintaining the rules of chess.

5.1.2 Convolutional Neural Networks

A Convolutional Neural Network (CNN) is a type of deep learning model designed to identify patterns in structured input data. Convolutional layers apply learnable filters to input representations and extract features from their spatial arrangement.

A chessboard contains pieces positioned across eight rows and eight columns. Representing the board numerically allows a neural network to process information about piece locations and other encoded board features. Convolutional layers can then learn patterns from this representation.

In the AI Chess Bot, the current board position is converted into a tensor with dimensions of $18 \times 8 \times 8$. This tensor is supplied to the neural network as input for move prediction.

5.1.3 Residual Neural Networks

Residual Neural Networks use skip connections that allow information to pass across multiple network layers. These connections help train deeper neural networks by providing an alternative path for information and gradients.

The project's policy network is described as containing an initial convolutional layer, ten residual blocks, a policy head, and an output layer. The residual blocks process features extracted from the encoded chessboard. The policy head converts the learned representation into scores associated with possible chess moves.

The model architecture allows the system to learn relationships between board positions and candidate moves. Its actual playing strength depends on the trained weights and the quality of the learned policy.

5.1.4 Policy Networks and Move Selection

A policy network predicts which actions are promising for a given state. In a chess application, the state is the current board position and the actions are possible chess moves.

The model produces scores or probabilities associated with its move representation. The move-selection component processes these predictions and identifies a suitable candidate from the legal moves available in the current position.

The implementation may apply temperature-based sampling and top-k selection to control how candidate moves are selected. Legal-move filtering is essential because a neural network prediction alone does not guarantee that a move follows chess rules.

The selected move is returned to the game-management component, which updates the board state and provides the resulting position to the frontend.

5.2 Methodological Procedure

The methodology describes the sequence of operations performed when a user plays against the AI Chess Bot. The system follows a modular workflow in which user input is processed by the backend, the board position is encoded, the neural network predicts candidate moves, and a legal AI move is applied to the game.

5.2.1 User Input and Move Validation

The process begins when the human player makes a move through the web interface. The frontend sends the move to the FastAPI backend through an API request.

The backend forwards the request to the game-management component. The game manager checks whether the submitted move is legal in the current board position. If the move is invalid, the application should reject it or return an appropriate error response. If it is valid, the game manager applies the move and updates the board state.

This validation step prevents invalid user actions from being applied to the game.

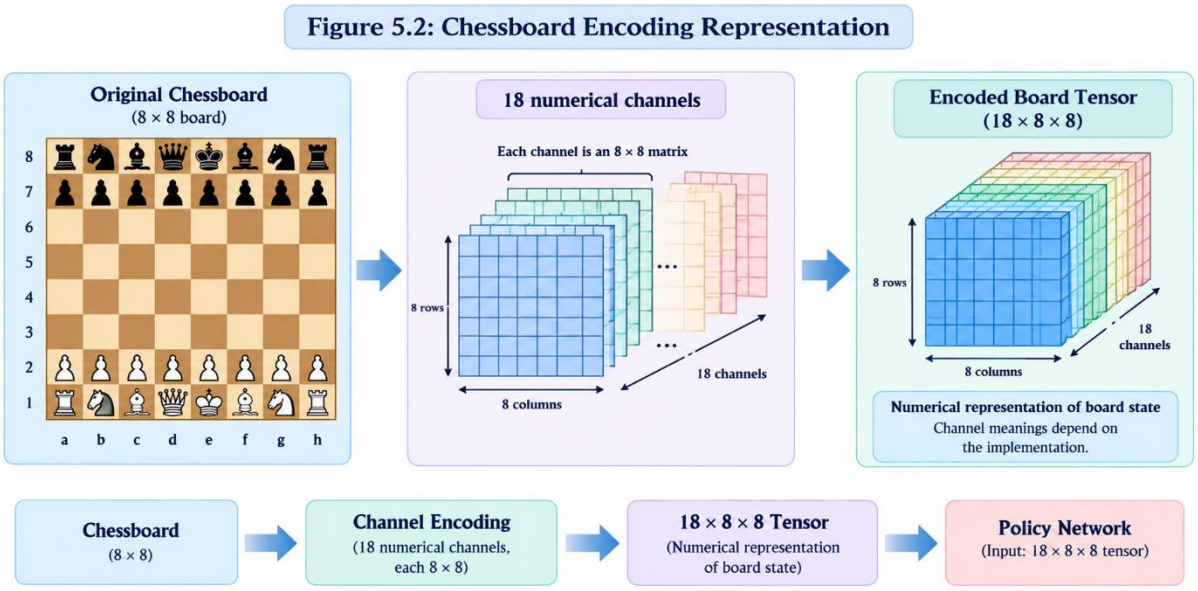
5.2.2 Chessboard Encoding

After a valid human move, the current board position is converted into a numerical tensor that can be processed by the neural network.

The board-encoding component represents the chessboard using 18 channels, with each channel containing an 8 × 8 grid. The resulting tensor has dimensions of 18 × 8 × 8.

The encoded representation is passed to the PyTorch policy network. This conversion provides the model with structured numerical input rather than a visual image of the chessboard.

Figure 5.2: Chessboard Encoding Representation



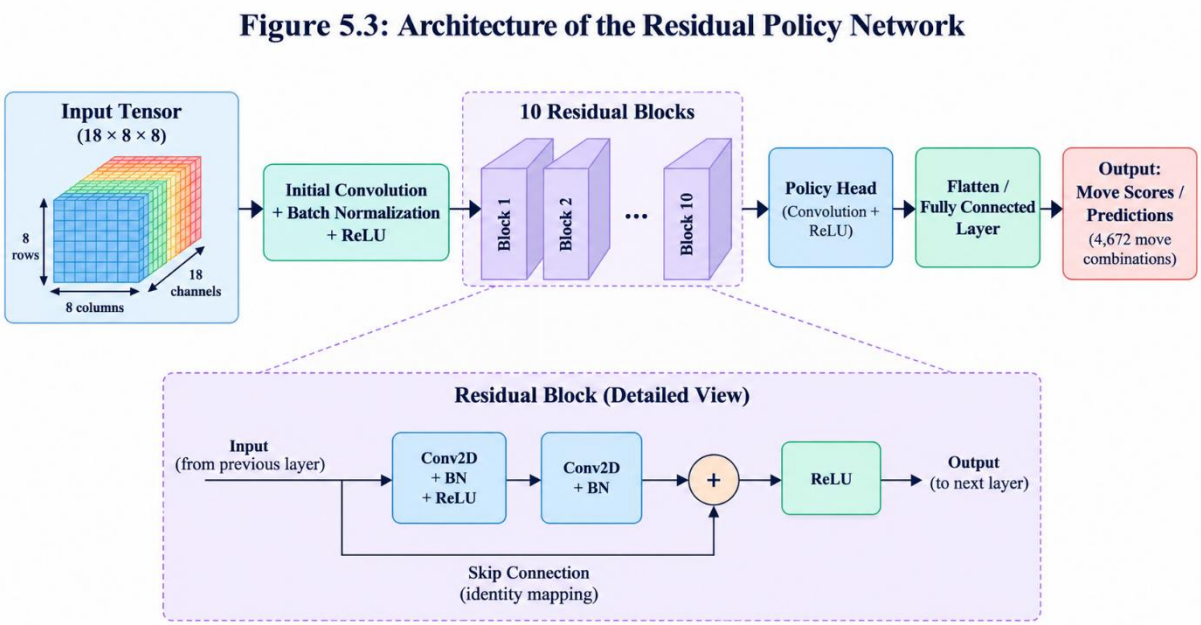
5.2.3 Neural Network Inference

The encoded tensor is passed to the trained PyTorch policy network. The network processes the input through its convolutional and residual layers and generates scores for the possible move representations.

The model's output representation contains 4,672 possible move combinations according to the project's described architecture. These outputs provide the move-selection component with candidate scores for the current position.

The network performs inference using the trained model weights. The quality of the predictions depends on the model's training and learned parameters. The inference component does not independently enforce all chess rules, so its predictions must be processed by the move-selection and game-management components.

Figure 5.3: Architecture of the Residual Policy Network



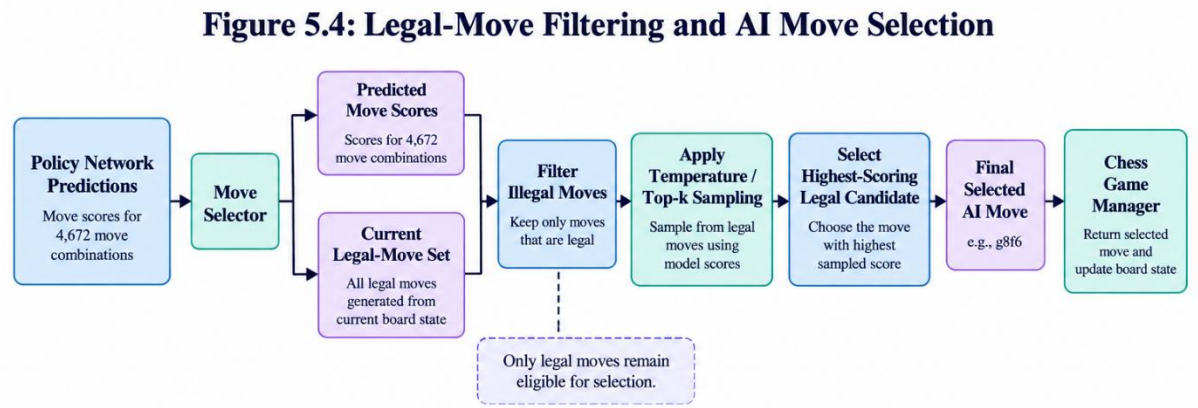
5.2.4 Legal-Move Filtering and AI Move Selection

The move-selection component receives the model predictions and the legal moves available in the current board position. It processes the predictions and selects a candidate move that is permitted by the chess rules.

Temperature-based sampling can adjust the relative influence of candidate scores, while top-k selection can restrict selection to a subset of candidates. The precise behaviour depends on the implemented selection procedure and configuration.

After selecting a move, the system returns it to the game manager. The game manager applies the AI move, updates the board, and prepares the updated game state for the frontend.

Figure 5.4: Legal-Move Filtering and AI Move Selection



5.2.5 Updated Board State and Response

After the AI move is applied, the game manager obtains the updated chessboard position. The backend then returns the relevant game-state information to the frontend.

The frontend displays the updated chessboard, allowing the user to observe the AI move and continue the game if it has not ended. The application should also handle supported game-ending conditions, such as checkmate or a draw, when detected by the chess rules component.

This process repeats until the game ends or the user stops playing.

5.3 Implementation Workflow

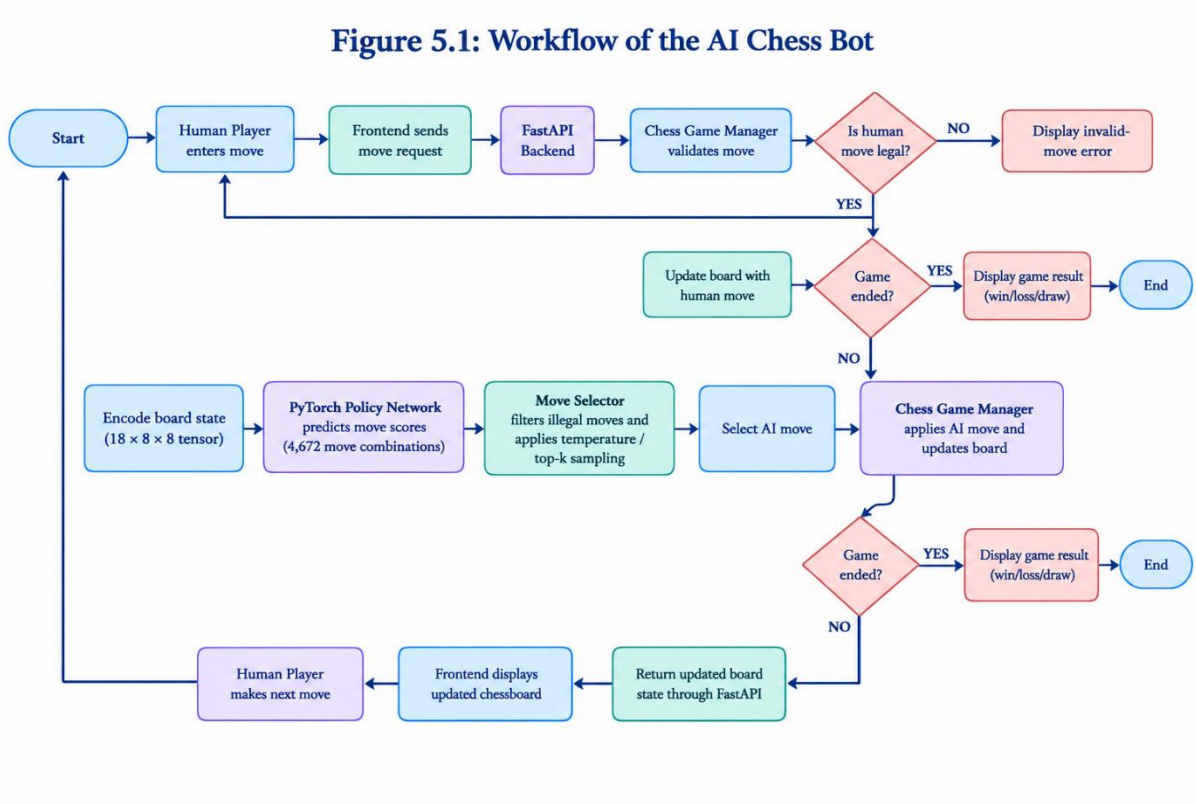
The implementation follows a modular development approach. Individual components are developed or configured and then connected to form the complete application.

The workflow begins with the frontend, which provides the chessboard and accepts user actions. The FastAPI backend handles incoming requests and coordinates operations with the game manager. The game manager maintains the board position and validates moves. When an AI response is required, the board encoder prepares the numerical representation, and the policy network generates predictions. The move selector processes the predictions, selects a legal move, and returns it to the game manager. Finally, the updated game state is sent to the frontend.

The major implementation stages are:

1. Set up the project environment and install the required dependencies.
2. Prepare the frontend interface and chessboard interaction.
3. Configure the FastAPI backend and request-handling logic.
4. Integrate the chess game-management component.
5. Connect the board encoder to the trained neural network.
6. Implement or configure the move-selection and legal-move filtering process.
7. Connect the AI response to the game manager and frontend.
8. Test individual modules and verify the complete request-response workflow.

Figure 5.1: Workflow of the AI Chess Bot



5.4 Module Description

The application is divided into modules to separate user interaction, API communication, game management, numerical encoding, model inference, and move selection. This organization makes the implementation easier to understand, test, and maintain.

Table 5.1: Module Description of the AI Chess Bot

Module	Responsibility	Input	Output
Frontend Module	Displays the chessboard and accepts user actions	User interactions and game-state responses	Move requests and displayed board state
FastAPI Backend	Handles application requests and responses	API requests from the frontend	Responses containing game-state information
Chess Game Manager	Maintains the board state and validates chess moves	Human moves and selected AI moves	Updated board state and legal-move information
Board Encoding Module	Converts the current board into a numerical tensor	Current chessboard position	$18 \times 8 \times 8$ tensor
Neural Network Module	Generates predictions for candidate moves	Encoded board tensor	Model scores or move predictions
Move Selection Module	Processes predictions and selects a legal candidate	Model predictions and legal moves	Selected AI move
Model Loading and Inference Support	Makes the trained model available for inference	Model weights and encoded board input	Predictions from the loaded model
Integration Module	Coordinates communication between the application components	Requests, predictions, and game-state updates	Complete gameplay response

The modules operate together during gameplay. The frontend and API manage communication, while the game manager controls the chess state. The encoding and neural network components generate predictions, and the move selector processes these predictions before the selected move is applied.

5.5 Testing

Testing is performed to verify that the application operates correctly and that its modules work together as intended. The testing process should cover individual functions, API communication, move legality, and complete gameplay.

The following testing categories are considered:

- **Functional testing:** Verifies that the main user-facing functions operate as expected.
- **API testing:** Checks request handling and response behaviour of the backend.
- **Move validation testing:** Verifies that legal moves are accepted and illegal moves are rejected.

- **Integration testing:** Checks whether the frontend, backend, game manager, and neural network operate together.
- **Performance testing:** Measures response times under defined test conditions.

The test results should be recorded after running the application. Expected results describe the intended behaviour, while actual results must be based on observations from the implemented system.

5.5.1 Functional Testing

Functional testing checks whether the application's principal features behave according to their requirements. The tests cover displaying the chessboard, processing human moves, generating AI responses, and updating the board.

Table 5.2: Functional Testing Test Cases

Test ID	Test Case	Actual Result	Status
FT1	Launch the application	Chessboard displayed successfully	Observed
FT2	Display the initial board	Pieces displayed in their initial positions	Observed
FT3	Display the turn indicator	Selected side is reflected in player indicator; Black start triggers the AI opening move	Observed
FT4	Make a legal human move	Move is accepted	Observed
FT5	Generate an AI response	AI generates a move	Observed
FT6	Display the updated board	Board updates after moves	Observed
FT7	Restart the game	Board resets to the initial position	Observed

5.5.2 API Testing

API testing verifies communication between the frontend and the FastAPI backend. The tests should check whether requests are processed correctly, whether responses contain the expected information, and whether invalid requests are handled appropriately.

The specific endpoint paths and request formats must match the actual API implementation.

Table 5.3: API Testing Test Cases

Test ID	Test Case	Expected Result	Actual Result	Evidence

AT1	Open the deployed application	Frontend loads successfully	Application loaded successfully in the browser	Code-reviewed
AT2	Submit a legal move through the interface	Backend processes the move request	Move request processed successfully	Code-reviewed
AT3	Request an AI response after a human move	Backend returns an AI move	AI responds with a move during gameplay	Code-reviewed
AT4	Return the updated board state	Updated board state is displayed in the frontend	Board updates correctly after moves	Code-reviewed
AT5	Restart the game	Application resets to the initial position	Restart functionality works correctly	Code-reviewed
AT6	Submit an invalid move request	Backend rejects the move or returns an appropriate error	Invalid move request handled correctly	Code-reviewed

5.5.3 Move Validation Testing

Move validation testing verifies whether the AI Chess Bot follows the legal rules of chess. The system must accept valid moves and reject illegal moves without incorrectly changing the current board position. This testing also checks whether the AI-generated move is valid for the current chessboard state.

The Python Chess library is responsible for managing chess rules and checking the legality of moves. The move-selection component uses the available legal moves to ensure that the selected AI move can be applied to the current position.

Table 5.4: Move Validation Testing Results

Test ID	**Test Scenario**	**Expected Result**	**Actual Result**	Evidence
MV1	Make a legal pawn move	Move is accepted	Legal pawn move accepted successfully	Code-reviewed
MV2	Attempt an illegal piece movement	Move is rejected	Illegal move rejected successfully	Code-reviewed
MV3	Attempt a move from an empty square	Move is rejected	Move from empty square rejected successfully	Code-reviewed
MV4	Attempt a move that leaves the king in check	Move is rejected	Move violating king safety rejected successfully	Code-reviewed

MV5	Check the AI-generated move	AI selects a legal move	AI generated a legal move successfully	Code-reviewed
MV6	Check the board after a rejected move	Board remains unchanged	Board state remained unchanged after rejection	Code-reviewed

5.5.4 Integration Testing

Integration testing verifies whether the frontend, FastAPI backend, chess game manager, board encoder, neural network, and move-selection component work together correctly. The purpose is to ensure that data passes between the modules without disrupting the game state.

The integration process begins when the frontend submits a human move to the backend. The game manager validates the move and updates the board. The board encoder prepares the numerical input for the neural network, which generates move predictions. The move selector processes the predictions and returns an AI move to the game manager. Finally, the updated board state is returned to the frontend.

Table 5.5: Integration Testing Results

Test ID	**Integration Scenario**	**Expected Result**	**Actual Result**	Evidence
IT1	Frontend loads the application	Chessboard is displayed	Chessboard displayed successfully	Code-reviewed
IT2	Frontend communicates with FastAPI	Move request is processed	Move request processed successfully	Code-reviewed
IT3	Game manager updates the board	Board state is updated correctly	Board state updated successfully	Code-reviewed
IT4	Board encoder prepares model input	Correct tensor dimensions are produced	Board tensor prepared successfully	Code-reviewed
IT5	Neural network generates predictions	Model produces move predictions	Move predictions generated successfully	Code-reviewed
IT6	Move selector returns an AI move	A legal AI move is selected	Legal AI move selected successfully	Code-reviewed
IT7	Updated state returns to the frontend	Updated chessboard is displayed	Updated board displayed successfully	Code-reviewed

IT8	Restart game functionality	Board returns to its initial position	Board reset successfully	Code-reviewed

5.5.5 Testing Evaluation

The testing process evaluates the functional correctness and integration of the AI Chess Bot. The results recorded in Tables 5.2–5.5 cover the loading of the chessboard, processing of user moves, AI response generation, board updates, and restart functionality. Additional verification is required for internal model dimensions and specific chess-rule edge cases.

The following performance measures can be used to evaluate the application after collecting the required test data.

1. Test Pass Rate

The test pass rate measures the percentage of executed test cases that pass.

Test Pass Rate (%) = (Number of Passed Test Cases / Total Executed Test Cases) × 100

2. Average Response Time

Average response time measures the average duration required to process a set of requests.

Average Response Time = Total Response Time for All Requests / Number of Requests

3. Legal-Move Success Rate

The legal-move success rate measures the percentage of tested AI moves that are legal in their corresponding board positions.

Legal-Move Success Rate (%) = (Number of Legal AI Moves / Total AI Moves Tested) × 100

These measures should be calculated using recorded test results. Response-time measurements should specify the hardware, test conditions, and whether the measurement covers model inference alone or the complete request-response cycle.

CHAPTER 6

6. PROJECT DEMONSTRATION

This chapter demonstrates the running interface using the screenshots available with the project. The captures show the initial board and an updated board after the AI response. API payloads and a completed game are not shown; those behaviors are explained from the implementation and identified as such.

The demonstration focuses on the initial chessboard, human move interaction, AI move generation, updated board-state display, and handling of game-ending conditions. Screenshots from the running application are used to illustrate the major features.

6.1 Initial Chessboard Interface

The initial interface is the first screen displayed when the user opens the AI Chess Bot application. It contains the application title, a turn indicator, the chessboard, and a Restart Game button. The chessboard displays the white and black pieces in their standard starting positions.

The turn indicator informs the user when it is their turn to play. The Restart Game button allows the user to return the board to the initial position and begin a new game. The graphical interface provides a straightforward way to interact with the application without directly accessing backend code.

Figure 6.1: Initial Chessboard Interface



6.2 Chessboard After a Human Move

After the game starts, the human player selects a chess move through the interface. The frontend sends the move to the backend, where the game-management component checks its legality and updates the current board position.

When a valid move is accepted, the interface displays the updated chessboard. The system then prepares the current position for AI move generation if the game is still in progress. This interaction demonstrates how user input is processed and connected to the backend game logic.

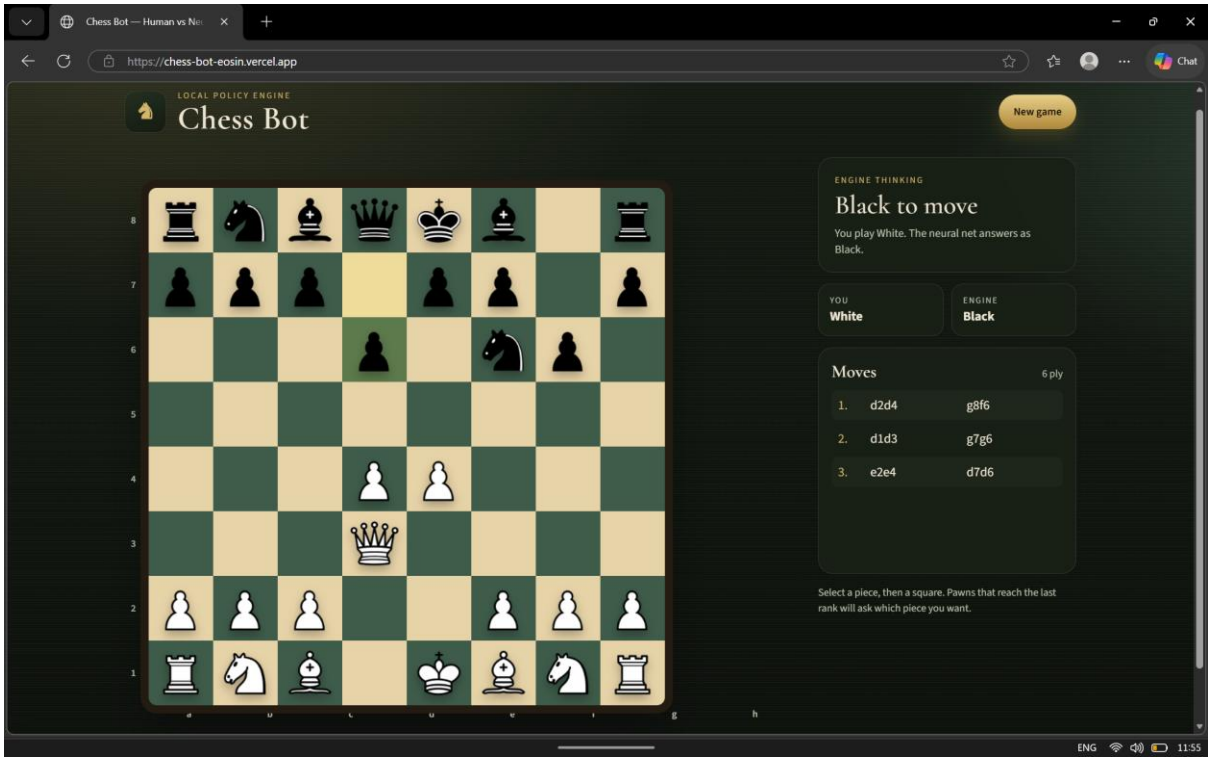
6.3 Chessboard After an AI Move

After a valid human move, the AI system processes the current board position to generate a response. The board-encoding component converts the position into a numerical tensor, which is passed to the PyTorch policy network. The model generates predictions for possible

moves, and the move-selection component processes these predictions to select a legal candidate.

The game manager applies the selected AI move and updates the board state. The frontend then displays the resulting position so that the human player can continue the game.

Figure 6.3: Chessboard After an AI Move



6.4 Game-State or API Response

The application communicates with the backend through API requests and responses. When a move is submitted, the backend processes the request and returns information required by the frontend to display the updated game state.

The response includes the game identifier, FEN position, move history, terminal status and result, check status, side to move, human and AI colors, and legal moves. The frontend uses this state to refresh the board and status indicators. A dedicated API-payload screenshot is not included.

6.5 Game-Over State Handling

python-chess supplies terminal outcomes, including checkmate and draws. The frontend reads the result and termination fields and displays a final status; checkmate sound and vibration feedback depend on browser support. No completed-game screenshot is included, so this behavior is documented from the implementation rather than claimed as a captured experiment.

The game manager prevents further moves once the position is terminal. The frontend displays the returned result and termination status. The current report includes no completed-game screenshot.

CHAPTER 7

7. RESULT AND DISCUSSION

This chapter presents the results and discussion of the AI Chess Bot developed using PyTorch, FastAPI, and deep learning. The application integrates a web-based chess interface with a backend game-management system and a neural network that generates AI move predictions. The results are discussed in terms of application functionality, AI move generation, move validation, gameplay behaviour, and overall system integration.

The deployed application provides a chessboard interface through which a human player can play against an AI opponent. The interface displays the board position, player information, game status, and move history. The application also supports starting a new game. The following sections discuss the observed functionality and the evaluation measures that can be used to assess the system.

7.1 Functional Testing Results

The following results summarize limited manual observations of the deployed application and source-level behavior. They are not results from a repeatable automated test suite. “Observed” means directly inspected; “Source-verified” means it follows from the implementation; “Not measured” means no empirical result is claimed. The project folder does not include a formal automated test suite or archived test run.

The application was observed to load its chessboard interface successfully. The initial position displays the chess pieces on the board, and the interface provides controls for interacting with the game. Based on the gameplay checks performed, human moves and AI responses are working, and the restart functionality returns the board to the initial position.

Table 7.1: Functional Testing Results

Test ID	Function Tested	Observed Result	Status
FT1	Application loading	Chessboard interface loaded successfully	Observed
FT2	Initial board display	Chess pieces displayed in their starting positions	Observed
FT3	Human move handling	Human moves processed during gameplay	Observed
FT4	AI move generation	AI generated responses during gameplay	Observed
FT5	Updated board display	Board updated after moves	Observed
FT6	Restart game	Game returned to its initial position	Observed

The functional tests indicate that the main user-facing features are operational in the tested environment. These observations demonstrate the successful integration of the interface and the gameplay flow. They do not, by themselves, establish the strategic strength of the AI model.

7.2 Response-Time Evidence and Measurement Plan

Response time is an important measure of the responsiveness of an AI-powered application. In this project, response time can be measured as the duration between submitting a valid human move and receiving the completed AI response at the frontend.

A reproducible timing dataset, hardware profile, and measurement procedure were not available with the project materials. Consequently, this report does not publish numeric latency results. A future benchmark should record repeated end-to-end durations, identify deployment and hardware conditions, and report median and spread alongside the sample count.

Table 7.2: AI Move Response-Time Measurements

Test ID	Test Scenario	Response Time (seconds)
—	No reproducible measurements or benchmark log supplied	Not measured

The response-time table records the evidence status for this project. No timing values are presented because underlying observations could not be reproduced from project records.

Recommended future metric: mean response time = sum of measured end-to-end durations / number of valid measurements. Report the median and range as well, because hosted inference latency can vary.

7.3 Legal-Move Validation Results

Legal-move validation ensures that the application follows the rules of chess. Human moves should be accepted only when they are legal in the current position, and the AI should select a move from the legal moves available to it.

The chess game-management component is responsible for maintaining the board state and validating moves. The move-selection component processes the neural network predictions and uses legal-move filtering to prevent an invalid candidate from being applied to the board.

Table 7.3: Legal-Move Validation Results

Test ID	Validation Scenario	Expected Behaviour	Evidence
MV1	Submit a legal human move	Move is accepted	Code-reviewed
MV2	Submit an illegal move	Move is rejected	Code-reviewed
MV3	Attempt a move from an empty square	Move is rejected	Code-reviewed
MV4	Check an AI-generated move	AI move is legal in the current position	Code-reviewed
MV5	Verify board after a rejected move	Invalid move does not alter the board	Code-reviewed

The results demonstrate the importance of combining neural network predictions with the chess rules component. The neural network proposes candidate moves, while the game-management and move-selection components ensure that the move applied to the board is legal.

7.4 Game Scenario Evaluation

Game scenario evaluation examines the application's behaviour across different gameplay situations. The purpose is to determine whether the system continues to function correctly as the board position changes and whether it handles relevant game conditions.

The following scenarios can be evaluated:

- Initial game setup and display of the starting board.
- Processing a legal human move.
- Generating and displaying an AI response.
- Updating the board after both players have moved.
- Restarting the game and returning to the initial position.
- Handling an invalid move or a supported game-ending condition.

Table 7.4: Game Scenario Evaluation

Scenario ID	Scenario	Expected Result	Status
GS1	Start a new game	Initial board is displayed	Observed

GS2	Human makes a legal move	Board state is updated	Observed
GS3	AI responds to a human move	AI move is displayed on the board	Observed
GS4	Continue gameplay over multiple turns	Game state remains consistent	Observed
GS5	Restart the game	Board returns to the starting position	Observed
GS6	Reach a game-ending condition	Supported game-ending state is handled correctly	Code-reviewed

The available manual observation confirms the initial board, selected-side start, and AI opening response. Other scenario rows report source-level behavior where labeled Code-reviewed. They do not establish complete-game reliability, special-move coverage, load performance, or playing strength.

7.5 Cost Analysis

The project uses software technologies such as Python, PyTorch, FastAPI, and the Python Chess library. These tools can be used without purchasing commercial software licences. Frontend development is based on HTML, CSS, and JavaScript.

The overall cost depends on the hardware used, internet connectivity, model hosting, and deployment services. Since the application is deployed through a web hosting platform, any applicable hosting or usage charges should be considered when calculating the actual cost.

Table 7.5: Cost Analysis

Component	**Description**	**Cost**
Python	Programming language	No software licence cost
PyTorch	Deep learning framework	No software licence cost
FastAPI	Backend API framework	No software licence cost
Python Chess	Chess rule-management library	No software licence cost
HTML, CSS, JavaScript	Frontend technologies	No software licence cost
Development Computer	Hardware used for development and testing	Existing hardware; purchase cost not included
Internet Connectivity	Used for development, testing, and deployment access	Depends on the user's plan
Web Hosting	Hosting the deployed application	Verify actual plan and charges

Model Hosting or Storage	Storage and retrieval of model weights, if applicable	Depends on the service and usage

The project can be developed with a low direct software cost because its core development technologies are available without commercial licence fees. However, the total cost should be calculated from actual expenses rather than assumed to be zero. Existing hardware, internet access, and any hosting services may have associated costs.

CHAPTER 8

8. CONCLUSION

8.1 Conclusion

The project, titled *AI Chess Bot Using PyTorch, FastAPI, and Deep Learning*, focuses on developing an interactive chess application that allows a human player to compete against an AI opponent. The system combines a web-based user interface, a FastAPI backend, chess game management, board-state encoding, and a trained PyTorch policy network to generate AI moves.

The application successfully demonstrates the main gameplay functions, including displaying the chessboard, processing human moves, generating AI responses, updating the board state, and restarting the game. The frontend provides a visual interface for playing chess, while the backend coordinates move processing and communication between the application components.

The board-encoding module converts the chess position into a numerical representation suitable for neural network inference. The policy network generates predictions for possible moves, and the move-selection component processes these predictions to select an appropriate legal move. This design combines deep learning-based predictions with chess-rule validation.

Manual gameplay inspection confirmed the board and selected-side start flow, including a Black start in which the AI made the opening move. Project files do not contain a reproducible benchmark log, so response-time claims are intentionally omitted. Broader claims about game-ending handling, special moves, reliability under load, and model playing strength require dedicated testing.

Overall, the project demonstrates how deep learning can be integrated with a web application and a rule-based game-management system to build an interactive AI chess-playing

application. It also provides practical experience in neural network inference, API development, frontend integration, and software testing.

8.2 Future Work

Although the current application demonstrates the core functionality of human-versus-AI chess gameplay, several improvements could be explored in future versions.

1. **Improved model training:** Train the policy network using a larger and more diverse chess dataset to improve the quality of its move predictions.
2. **Stronger chess decision-making:** Explore combining the policy network with search algorithms to evaluate promising move sequences before selecting a move.
3. **Performance optimization:** Optimize model inference and application communication to reduce response times and improve responsiveness.
4. **Difficulty settings:** Introduce configurable playing strengths so that users with different skill levels can play against the AI.
5. **Enhanced user interface:** Add features such as move highlighting, captured-piece displays, game statistics, and improved feedback for invalid moves.
6. **Game analysis:** Provide explanations of selected moves and identify possible mistakes or alternative moves.
7. **Comprehensive testing:** Evaluate the application across more board positions, special chess rules, longer games, and different computing environments.
8. **Deployment improvements:** Strengthen error handling, logging, monitoring, and deployment reliability to support a more robust application.

These improvements could make the AI Chess Bot more capable, informative, and user-friendly while providing opportunities for further experimentation with deep learning and chess decision-making.

CHAPTER 9

9. REFERENCES

[1] PyTorch Contributors, “PyTorch Documentation.” [Online]. Available: https://docs.pytorch.org/docs/stable/. [Accessed: Oct. 27, 2026].

[2] S. Ramírez, “FastAPI Documentation.” [Online]. Available: https://fastapi.tiangolo.com/. [Accessed: Oct. 27, 2026].

[3] N. A. F. R. van der Meijden, “python-chess: A Chess Library for Python.” [Online]. Available: https://python-chess.readthedocs.io/en/stable/. [Accessed: Oct. 27, 2026].

[4] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778, doi: 10.1109/CVPR.2016.90.

[5] D. Silver et al., “Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm,” arXiv:1712.01815, 2017. [Online]. Available: https://arxiv.org/abs/1712.01815.

[6] MDN Contributors, “HTML: HyperText Markup Language.” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTML. [Accessed: Oct. 27, 2026].

[7] MDN Contributors, “CSS: Cascading Style Sheets.” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS. [Accessed: Oct. 27, 2026].

[8] MDN Contributors, “JavaScript Guide.” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide. [Accessed: Oct. 27, 2026].

[9] Python Software Foundation, “Threading — Thread-based parallelism,” Python 3 Library Reference. [Online]. Available: https://docs.python.org/3/library/threading.html. [Accessed: Oct. 27, 2026].

[10] Hugging Face, “Downloading files,” Hugging Face Hub documentation. [Online]. Available: https://huggingface.co/docs/huggingface_hub/guides/download. [Accessed: Oct. 27, 2026].

[11] V. Raju, “Chess-Bot,” GitHub repository. [Online]. Available: https://github.com/VishnuRaju06/Chess-Bot. [Accessed: Oct. 27, 2026].

[12] V. Raju, “Chess-Bot,” Hugging Face Space. [Online]. Available: https://huggingface.co/spaces/vishnuraju06/Chess-Bot. [Accessed: Oct. 27, 2026].

APPENDIX A – SAMPLE CODE

A.1 Introduction

This appendix presents selected code snippets related to the implementation of the AI Chess Bot using PyTorch, FastAPI, and Python Chess. The snippets illustrate the main operations involved in processing chess moves, validating moves, generating AI responses, and communicating between the frontend and backend.

A.2 Chess Move Validation

The game manager parses the UCI move, verifies that it is the human player’s turn, and checks it against the current legal-move set before changing the board. Invalid requests are returned by the API as HTTP 400.

```
move = chess.Move.from_uci(move_uci)
if board.turn != human_color:
    raise ValueError("Not human turn")
if move not in board.legal_moves:
    raise ValueError("Illegal move")
board.push(move)
```

A.3 Board-State Processing

The encoder creates a float32 tensor. Piece planes are followed by side-to-move, castling-right, and fullmove-number channels.

```
tensor = np.zeros((18, 8, 8), dtype=np.float32)
# Piece planes 0–5: White; 6–11: Black
tensor[12, :, :] = int(board.turn)
# Channels 13–16: castling rights
tensor[17, :, :] = board.fullmove_number / 100.0
```

A.4 AI Move Generation

Inference runs without gradient tracking. Candidate policy scores are restricted to python-chess legal moves before selection.

```
with torch.no_grad():
    logits = model(x)[0].cpu().numpy()
legal = list(board.legal_moves)
indices = [m.from_square * 64 + m.to_square for m in legal]
scores = logits[indices]
# Select only among legal candidates
```

A.5 FastAPI Backend

Each new game receives a unique identifier and its own ChessGame instance and re-entrant lock. Game mutations are serialized per game while the model is shared.

```
game_id = uuid4().hex
new_game = ChessGame(engine)
new_game.reset(human_color=req.color)
game_lock = RLock()
with games_lock:
    games[game_id] = new_game
    game_locks[game_id] = game_lock
```

A.6 Summary

The code snippets illustrate important operations involved in the AI Chess Bot, including move validation, board-state processing, AI move application, and backend API communication. Together, these components support interaction between the user interface, chess game logic, and AI inference system.